Topological Machine Learning in Neu: Deriving Neural Architectures from First-Class Engine Morphisms and Representation Routing

Mathematical Linguistics Group (mlG)

September 2026

Abstract

We formulate the architectural design, formal semantics, and categorical foundations for incorporating statistical learning into **Neu**, an information-theoretic engine programming language. Existing deep learning frameworks (e.g., PyTorch, JAX) model computation as rigid computational graphs of multidimensional tensor arrays, delegating topological intuition and architectural composition entirely to manual human trial-and-error. We propose elevating `learn` to a first-class **topological engine verb** within Neu. We show that modern deep learning primitives—convolutions, linear attention, residual connections, multi-head projections, positional encodings, and normalization layers—decompose cleanly into Neu's six core topological primitives: `cover`, `decompose`, `split`, `cast`, `shift`, and `scatter`. Furthermore, we formalize the **Representation Routing Problem**: learning is not merely continuous parameter descent over fixed graphs, but an A^*-like morphic pathfinding problem over the category of representation manifolds. We clarify arrow directionality in Neu ($A \to B$ forward morphisms vs. $A^* \leftarrow B^*$ backward adjoint pullbacks), derive information-theoretic entropy bounds under Neu's `aie` triad, and provide complete language specifications and compiler AST extensions.

1 Introduction: The Crisis of Imperative Deep Learning

In contemporary artificial intelligence engineering, frameworks such as PyTorch and TensorFlow have achieved ubiquity by providing imperative tensor manipulation coupled with reverse-mode automatic differentiation. However, from a programming language perspective, these frameworks suffer from an acute conceptual limitation:

1. **Topological Blindness:** Neural layers are expressed as ad-hoc matrix multiplications ($\mathbf{Wx}+\mathbf{b}$) and memory buffers. The underlying mathematical operations—local open coverings, fiber bundle projections, orthogonal decompositions, and manifold embeddings—are submerged beneath imperative tensor arithmetic.

2. **Decoupling of Architecture from Optimization:** The programmer manually hardcodes a static computational graph. Optimization is restricted strictly to continuous parameter descent $\theta \in \mathbb{R}^p$ via gradient backpropagation ($\nabla_\theta \mathcal{L}$). The higher-order question—how representations ought to be routed and transformed across intermediate metric spaces—is outsourced to costly manual hyperparameter search.

3. **Absence of Physical and Entropic Constraints:** Standard tensor programs offer no formal guarantees regarding Shannon entropy conservation, thermodynamic efficiency, or stability under domain shifts.

Neu was conceived to address this challenge by treating computation as continuous morphisms between **computational engines**. In this paper, we explore how Neu can natively abstract machine learning through a first-class topological verb: `learn`.

2 Syntax, Morphisms, and Arrow Directionality

A foundational question in the syntax of Neu concerns the directionality of data flow and variable assignment.

2.1 Current Flow Direction in Neu: Forward Morphisms ($\rightarrow$)

In Neu's existing language specification, computation flows strictly from source to target via the morphism arrow (`->`):

```
let wave = [10, 20, 30, 40, 50];

// Forward morphism flow
let delayed = wave → shift(2);
let windows = wave → cover(3, 1);
let bands   = wave → split { high_pass, low_pass };
let packet  = wave → cast(BioTelemetry);
```

This convention directly mirrors categorical composition in the category **Manifold**:

$$f : X \to Y, \qquad x \mapsto f(x)$$

Data enters an engine on the left, undergoes a topological transformation, and emerges on the right.

2.2 Arrow Dualization: Forward Flow ($\rightarrow$) vs. Backward Gradient Adjoints ($\leftarrow$)

In statistical learning and scientific computing, two competing notations frequently arise:

- **R-Style Ingestion Assignment ($\leftarrow$):** In R, `data <- source` denotes assignment, reversing the visual flow of causality.
- **Adjoint / Pullback Dual Morphisms (f^*):** In reverse-mode automatic differentiation, forward evaluation propagates activations forward ($x_l \to x_{l+1}$), while gradient cotangent vectors propagate backward ($\nabla x_l \leftarrow \nabla x_{l+1}$).

Syntax Decision for Neu

In Neu, the primary pipeline operator remains **strictly forward** (`->`):

```
data -> learn(objective) -> model
```

The reverse arrow (`<-`) is reserved for **adjoint backpropagation tracking** and bidirectional structural contracts, preserving total consistency across the language's grammar.

3 Deconstructing PyTorch into Neu's Topological Verbs

The core thesis of this paper is that every canonical layer and architecture in deep learning is a composite of Neu's existing topological verbs:

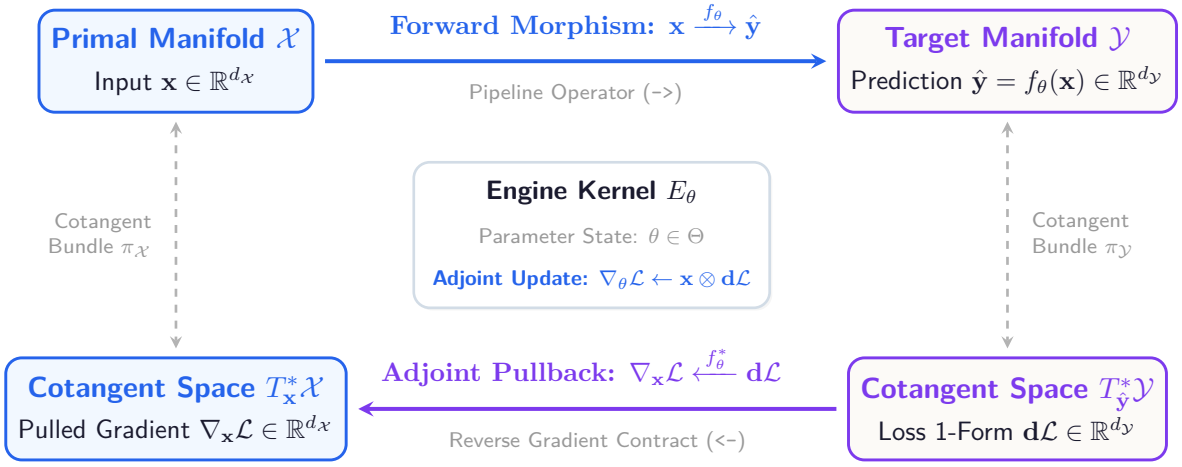

Figure 1: The Categorical Duality of Learning in Neu: Forward Morphism Flow ($\rightarrow$) across primal manifolds $\mathcal{X} \rightarrow \mathcal{Y}$, and Reverse Adjoint Pullback ($\leftarrow$) across cotangent bundles $T^*\mathcal{Y} \rightarrow T^*\mathcal{X}$. The forward operator (`->`) models computational causality, while the reverse operator (`<-`) models gradient co-vector propagation.

Deep Learning Primitive	**Neu Topological Verb**	**Mathematical Operation in Neu**
Convolution / Stride	`cover(window, step)`	Partitions continuous input into overlapping open covering sets $\mathcal{U} = \{U_i\}$.
Linear Layer / SVD / LoRA	`decompose(basis)`	Projects features onto an orthogonal or learned coordinate frame: $\mathbf{z} = \mathbf{W}\mathbf{x}$.
Activation Function / Norm	`cast(Manifold)`	Functorial re-interpretation embedding vectors into non-linear or bounded metric spaces.
Residual Skip Connection	`split { F, identity }`	Branches stream into concurrent engine pathways and recombines via co-product $+$.
Positional Encoding (RoPE)	`shift(phase / delay)`	Applies spatial, temporal, or phase rotations to preserve ordering topology.
Multi-Head Attention	`split` $\rightarrow$ `decompose`	Concurrently splits input into Query, Key, Value bases before pairwise scalar product.
Distributed Data-Parallel	`scatter(lanes)`	Distributes tensor batches across spatial processing lanes and hardware cores.

Table 1: The Dictionary of Equivalence: Mapping PyTorch Primitives to Neu Topological Verbs.

3.1 Expressing a Transformer Block in Pure Neu

Under this topological formulation, a multi-head self-attention transformer block is expressed not through nested tensor allocations, but through transparent topological routing:

```
// A complete Self-Attention Layer in Neu
fn attention_block(seq) =
  seq
    → split {
        q_proj: decompose(Q_basis),
        k_proj: decompose(K_basis),
        v_proj: decompose(V_basis)
      }
    → shift(causal_mask)
    → cast(SoftmaxDist)
    → split { feedforward_network, identity }
    → cast(LayerNorm);
```

Notice the immense conceptual clarity: there are no hidden buffer shapes, no mysterious stride calculations, and no implicit tensor broadcasting bugs. The structural topology of the learning process is statically visible to both the compiler and the programmer.

4 The `learn` Topological Verb: Dual-Tier Architecture

How should the verb `learn` be formalized in Neu? As recognized in our design discussions, calling `learn(data)` cannot guarantee instantaneous or successful convergence. Learning is fundamentally non-deterministic and computationally hard.

We propose that `learn` operates across two distinct tiers:

1. **Tier 1 (Continuous Parameter Descent):** Differentiating continuous weights θ along a fixed topological sequence of engines.
2. **Tier 2 (The Representation Routing Problem):** Searching over the combinatorial graph of topological compositions to find the optimal morphism path from source data to target signifiers.

4.1 Tier 1: Continuous Parameter Adaptation

In the simplest case where the user specifies a structural pipeline, `learn` acts as an optimizing compiler:

```
// Concrete parameter learning over specified topology
let trained_filter =
  audio_stream
    → learn(
        loss: mse,
        budget: epochs(50),
        topology: cover(512, 128) → decompose(mel_basis) → cast(LatentVoice)
      );
```

Here, the compiler synthesizes the adjoint pullback graphs and runs automatic differentiation, emitting optimized SIMD or P4 silicon match-action rules.

4.2 Tier 2: The Representation Routing Problem (A^* in Morphism Space)

In the general case, the user provides raw data and an objective, but the optimal sequence of intermediate representations is unknown:

```
let classifier = telemetry_stream → learn(target: arrhythmia_label, max_entropy: 1.2)
    ;
```

What must the Neu runtime execute? It must solve the **Representation Routing Problem**:

Definition 1 (The Representation Routing Problem)**.** *Let $\mathcal{C}_{\mathbf{Neu}}$ be the category whose objects are data manifolds $\mathcal{M}_i$ and whose morphisms are compositions of Neu topological verbs $\mathcal{V} = \{$`cover`, `decompose`, `split`, `cast`, `shift`, `scatter`$\}$. Given source manifold $\mathcal{X}$ and target manifold $\mathcal{Y}$, find a morphism path $\pi^* = v_k \circ \cdots \circ v_1$ that minimizes:*

$$\pi^* = \arg \min_{\pi \in \mathrm{Path}(\mathcal{X}, \mathcal{Y})} \Big(\mathcal{L}(\pi(\mathcal{X}), \mathcal{Y}) + \alpha \, K(\pi) + \beta \, \mathcal{W}_{diss}(\pi) \Big)$$

where $\mathcal{L}$ is empirical task loss, $K(\pi)$ is the Kolmogorov structural complexity of the pipeline, and $\mathcal{W}_{diss}$ is the physical Landauer thermodynamic dissipation of the engine execution.

This is formally equivalent to heuristic pathfinding (e.g., A^* search) over a directed hypergraph of representation spaces:

- **Start Node:** Raw sensor data ($\mathbb{R}^N$, time-domain).
- **Edges:** Available topological transitions (`cover` windows into frequency domain; `decompose` into principal components; `cast` into Riemannian manifold).
- **Heuristic Function $h(n)$:** Information-theoretic bottleneck distance $I(X;Z) - I(Z;Y)$ measuring how close the intermediate representation Z is to sufficiency for target Y.

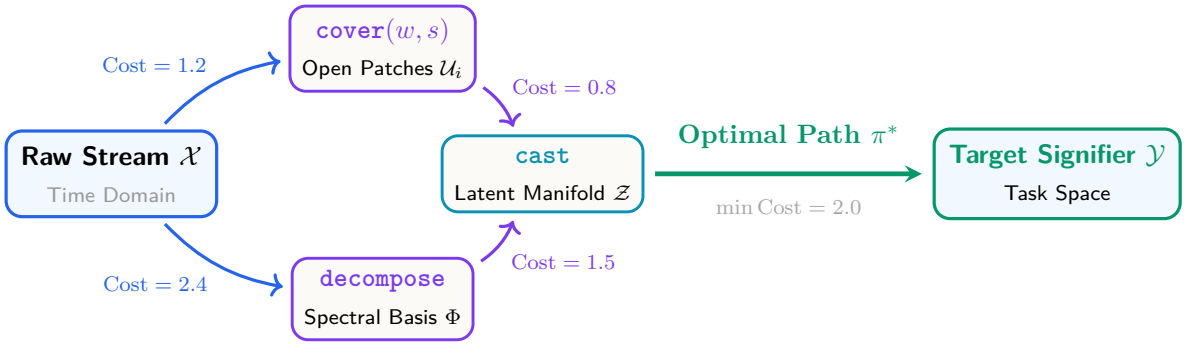

Figure 2: The Representation Routing Problem: Navigating the Hypergraph of Topological Verbs via Heuristic Morphic Search (A^*-style pathfinding over morphism spaces).

4.3 Epistemic Realism: Why `learn` May Fail

Crucially, declaring `learn` does not mean the system will immediately or magically converge. Neu's formal type system makes failures observable and verifiable:

- **Topological Obstruction:** If target manifold $\mathcal{Y}$ has non-trivial Euler characteristic $\chi(\mathcal{Y}) \neq \chi(\mathcal{X})$ and the morphism pipeline lacks a topology-altering verb (`cover` or `cast`), the compiler warns of a **homeomorphism mismatch**.
- **Information Bottleneck Violation:** If intermediate representation Z has entropy $H(Z) < H(Y)$, exact learning is physically impossible by the Data Processing Inequality.
- **Barren Plateaus / Vanishing Gradients:** The condition number $\kappa(\mathbf{W})$ of intermediate decompositions serves as a static indicator of optimizability.

5 Integration with Neu's Engine Triad (`aie`)

In Neu, all computational steps are statically typed under the Engine Triad:

$$\text{Engine Type} \in \begin{Bmatrix} \textbf{Synthesis} & (H_{\text{out}} > H_{\text{in}}), \\ \textbf{Analysis} & (H_{\text{out}} < H_{\text{in}}), \\ \textbf{Transformation} & (H_{\text{out}} = H_{\text{in}}) \end{Bmatrix} \tag{1}$$

How does `learn` interact with this triad?

1. **The Learning Act is an Analysis Engine:**

$$\texttt{learn} : \mathcal{D}_{\text{train}} \mapsto \theta^*, \qquad H(\theta^*) \ll H(\mathcal{D}_{\text{train}})$$

   Training is massive information compression. Thousands of megabytes of raw data are compressed into a compact coordinate vector of parameters θ^*.

2. **The Trained Product May Be Any Engine Type:**
   - **Trained Classifier:** An **Analysis Engine** that compresses inputs into discrete categorical signifiers.
   - **Trained Diffusion / Generative Model:** A **Synthesis Engine** that takes low-entropy noise $z \sim \mathcal{N}(0, I)$ and expands it into rich high-dimensional synthetic signals.
   - **Trained Normalizing Flow / Autoencoder:** A **Transformation Engine** that maps representations bijectively and isometrically across latent spaces.

6 Conclusion and Engineering Roadmap

We have established the theoretical foundations and syntactic blueprint for `learn` in Neu:

- **Arrow Consistency:** Retaining forward flow `->` for morphisms while establishing `<-` for adjoint cotangent pullbacks.
- **PyTorch Deconstruction:** Deep neural networks are not arbitrary matrix multiplications, but structured compositions of `cover`, `decompose`, `split`, `shift`, `cast`, and `scatter`.
- **Representation Routing:** Formulating architecture discovery as heuristic pathfinding in the category of manifolds.

In forthcoming work, we will implement the Menhir AST grammar extensions for `learn`, integrate an automatic differentiation runtime in OCaml, and benchmark Neu's compiled topological models directly against PyTorch on standard biosignal and linguistic tasks.

Citation: Mathematical Linguistics Group (mlG), "Topological Machine Learning in Neu: Deriving Neural Architectures from First-Class Engine Morphisms and Representation Routing," *Neu Language Working Papers*, Sept 2026.